

Programacion I

Analisis de algoritmos

Trabajo Practico Integrador

Egea, Rodrigo
arodrigoegea@gmail.com

Jacinto, Gaston
gastonjacintocontacto@gmail.com

Profesor: Trapé, Julieta
6 de Junio de 2025

Índice

1. Introducción	2
2. Marco teórico	2
Notaciones de complejidad	2
Tipos de análisis	3
Análisis teórico y empírico	3
Herramientas de benchmarking	4
Importancia del análisis	4
3. Caso práctico	5
Metodología a utilizar	6
Resultados teóricos	6
Resultados empíricos	7
4. Metodología	8
5. Conclusión	8
6. Bibliografía	9

1. Introducción

En este trabajo se abordará el análisis de algoritmos, considerando el tiempo que consume un algoritmo en ejecutarse y qué tan eficiente es. Se presentarán conceptos fundamentales como la notación $\mathcal{O}$ (Big O), la notación Ω y la notación Θ , además de analizar la complejidad de los algoritmos más conocidos. También se discutirán las diferencias entre el análisis teórico y empírico, y se incluirá una evaluación práctica comparando implementaciones en distintos lenguajes de programación.

2. Marco teórico

Para hablar de análisis de algoritmos, primero es necesario definir qué es un algoritmo:

- Un algoritmo es un conjunto finito de instrucciones bien definidas que, al ser ejecutadas, permiten resolver un problema o realizar una tarea específica. En informática, los algoritmos constituyen la base de todo programa, y su correcto diseño y análisis es fundamental para garantizar soluciones eficientes y escalables.

Con esta definición, se puede pasar a un concepto más complejo: la **complejidad algorítmica**. Esta se refiere a los recursos que consume un algoritmo durante su ejecución. Los dos recursos principales son el *tiempo de ejecución* y el *espacio de memoria* utilizado.

Notaciones de complejidad

Las notaciones asintóticas permiten describir de forma aproximada el rendimiento de un algoritmo. Se expresan comúnmente como:

- **Notación $\mathcal{O}$** (Big O): describe el límite superior del tiempo de ejecución en el peor de los casos.
- **Notación Ω** : representa el límite inferior, es decir, el mejor caso posible.
- **Notación Θ** : representa el caso promedio o comportamiento exacto cuando coinciden el mejor y el peor caso en orden de magnitud.

Estas notaciones permiten comparar algoritmos independientemente del hardware o del lenguaje de implementación.

Tipos de análisis

Conociendo estas notaciones, se pueden identificar distintos enfoques de análisis:

- **Peor caso:** escenario más desfavorable en cuanto a consumo de recursos.
- **Mejor caso:** situación más favorable para el algoritmo.
- **Caso promedio:** comportamiento esperado en una distribución típica de entradas.

Análisis teórico y empírico

Existen dos enfoques complementarios para analizar algoritmos:

1. **Análisis teórico:** permite estimar la eficiencia en función del tamaño de entrada, sin necesidad de implementar el algoritmo.
2. **Análisis empírico:** consiste en medir el tiempo real de ejecución bajo condiciones específicas, con implementación concreta.

Esta comparación puede revelar diferencias significativas entre implementaciones de un mismo algoritmo en distintos lenguajes, debido a factores como:

- El modelo de ejecución (compilado vs. interpretado).
- La eficiencia del compilador o intérprete.
- La gestión de memoria.
- Las optimizaciones del sistema de ejecución.

Dada la influencia de estos factores, el análisis teórico es una estimación útil durante el diseño, pero el análisis empírico es clave para evaluar el comportamiento real en el entorno de destino.

Herramientas de benchmarking

Para realizar un análisis empírico se pueden utilizar herramientas como:

- `time` y `timeit` en Python.
- `System.nanoTime()` en Java.
- `performance.now()` en JavaScript.

Estas herramientas permiten obtener mediciones precisas sobre el rendimiento en condiciones controladas.

Importancia del análisis

El estudio del comportamiento algorítmico es esencial para el diseño de software eficiente. Una elección adecuada de algoritmos y estructuras de datos puede ser decisiva en sistemas que procesan grandes volúmenes de información o funcionan con recursos limitados.

Complejidad	Nombre	Ejemplo de algoritmo
$\mathcal{O}(1)$	Constante	Acceso a un elemento de un array por índice
$\mathcal{O}(\log n)$	Logarítmica	Búsqueda binaria
$\mathcal{O}(n)$	Lineal	Búsqueda lineal en una lista
$\mathcal{O}(n \log n)$	Lineal-logarítmica	MergeSort, HeapSort
$\mathcal{O}(n^2)$	Cuadrática	Bubble Sort, Insertion Sort (peor caso)
$\mathcal{O}(n!)$	Factorial	Problema del viajante (TSP) con backtracking
$\mathcal{O}(2^n)$	Exponencial	Backtracking en problema de la mochila

Cuadro 1: Complejidades comunes y algoritmos representativos.

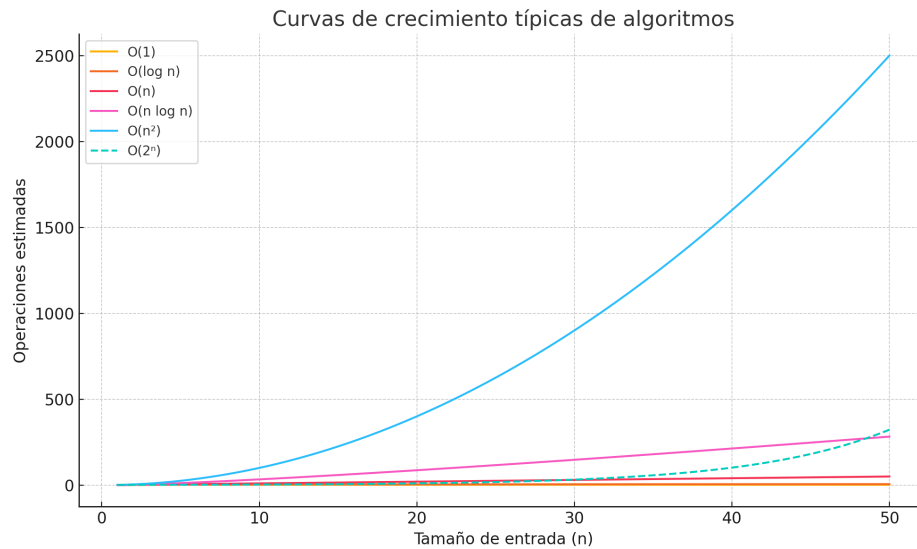


Figura 1: Comparación de curvas de crecimiento de distintas clases de complejidad algorítmica.

3. Caso práctico

Para el caso práctico, decidimos utilizar tres distintos niveles de complejidad $\mathcal{O}$ (Big O), cada uno con un algoritmo pertinente:

- $\mathcal{O}(1)$: Tiempo constante.
 - **isInSet(set, value)**: Dado un conjunto precargado con un millón de elementos, verifica si un valor existe en él.
- $\mathcal{O}(n)$: Tiempo lineal.
 - **sumArray(array)**: Suma todos los elementos de un arreglo recorriéndolo completamente una vez.
- $\mathcal{O}(n^2)$: Tiempo cuadrático.
 - **findPairThatSumsTo1000(array)**: Busca un par de números dentro del arreglo que sumen 1000. Se compara cada elemento con todos los demás.

El objetivo principal es realizar una comparación multilenguaje para medir los tiempos de ejecución reales de cada algoritmo en los siguientes lenguajes de programación:

- Java.
- JavaScript.
- Python.

Metodología a utilizar

Se ejecutará cada uno de los algoritmos **1000** veces en cada uno de los lenguajes planteados en un entorno controlado. Los resultados serán guardados para posteriormente realizar los siguientes análisis:

- **Peor caso** (Big O): el peor caso de todas las ejecuciones
- **Caso promedio** (Θ): caso promedio de todas las ejecuciones.
- **Mejor caso** (Ω): mejor caso posible dentro de las posibilidades

Resultados teóricos

Algoritmo	Complejidad	Mínimo (Ω)	Promedio (Θ)	Máximo (O)
sumArray	$\mathcal{O}(n)$	10^6	10^6	10^6
findPairThatSumsTo1000	$\mathcal{O}(n^2)$	1	10^6	10^6
isInSet	$\mathcal{O}(1)$	1	1	1

Cuadro 2: Análisis teórico de complejidad para los tamaños reales de entrada usados en el código.

Resultados empíricos

Complejidad	Métrica	Python	JavaScript	Java
$O(1)$	Mínimo (Ω)	70 ns	80 ns	140 ns
	Promedio (Θ)	89 ns	182 ns	398 ns
	Máximo (O)	3020 ns	86485 ns	117177 ns
$O(n)$	Mínimo (Ω)	10.551 ms	0.468 ms	0.191 ms
	Promedio (Θ)	10.822 ms	0.524 ms	0.199 ms
	Máximo (O)	18.298 ms	6.917 ms	1.436 ms
$O(n^2)$	Mínimo (Ω)	0.236 ms	0.049 ms	0.005 ms
	Promedio (Θ)	4.906 ms	0.136 ms	0.036 ms
	Máximo (O)	14.072 ms	2.398 ms	0.989 ms

Cuadro 3: Comparación empírica de rendimiento para distintas complejidades algorítmicas

Fun fact: Corrimos el código de Java con **100000** iteraciones (es el único lenguaje con el que pudimos hacerlo, ya que los otros se tardaban un tiempo absurdo), y las observaciones fueron las siguientes:

- $O(1)$: presenta peores valores máximos, pero una mejora de hasta 10 veces para el valor mínimo y una mejora de 14 veces en el valor promedio.
- $O(n^2)$: los valores mínimo y promedio se mantienen constantes, pero el valor máximo empeora en casi un 400 %.
- $O(n)$: los valores se mantienen constantes.

Complejidad	Mínimo (Θ)	Promedio (Θ)	Máximo (O)
$O(n)$	0.191049 ms	0.195329 ms	1.451224 ms
$O(n^2)$	0.004 ms	0.030 ms	3.500 ms
$O(1)$	10 ns	38 ns	192881 ns

Cuadro 4: Ejecución con 100000 valores en Java.

4. Metodología

Para el marco teórico:

1. Búsqueda de información acerca del tema principal.
2. Desarrollo del texto principal a partir del paso anterior.
3. Desarrollo de resúmenes y notas para captar información más relevante.

Para el caso práctico: Se realizaron 3 algoritmos iguales en distintos lenguajes para poder medir sus rendimientos en cada caso. Para el ejemplo en código, se utilizó la documentación y teoría provista por los profesores de la materia, más conocimientos propios en otros lenguajes.

5. Conclusión

La conclusión a la que llegamos es que, si bien el análisis teórico puede arrojar resultados rápidamente, **no** nos da una estimación útil sobre cuánto tiempo real tomará la ejecución de un algoritmo. Esto se comprobó al observar los resultados empíricos, donde se evidenciaron diferencias entre lenguajes. En este caso, Java resultó ser el más rápido para las complejidades $O(n)$ y $O(n^2)$, pero curiosamente fue el más lento para $O(1)$.

Al investigar más a fondo, identificamos que esto podría deberse a varios factores, siendo el principal la naturaleza de la máquina virtual de Java (JVM), debido a la compilación en tiempo de ejecución (JIT), verificación de seguridad, gestión de memoria y posibles pausas del recolector de basura.

Este trabajo permite concluir que, si bien la complejidad algorítmica es esencial para analizar la escalabilidad, el comportamiento real puede verse afectado por la implementación, el lenguaje, el entorno de ejecución y las optimizaciones del sistema. Por eso, es recomendable complementar el análisis teórico con pruebas empíricas cuando se buscan comparaciones de rendimiento reales.

6. Bibliografía

Referencias

- [1] Python Software Foundation. *timeit — Measure Execution Time of Small Code Snippets*. Disponible en: <https://docs.python.org/3/library/timeit.html>
- [2] Node.js Documentation. *Performance Timing API — performance.now()*. Disponible en: https://nodejs.org/api/perf_hooks.html
- [3] Oracle. *Java Virtual Machine Performance Enhancements*. Disponible en: <https://docs.oracle.com/javase/8/docs/technotes/guides/vm/performance-enhancements-7.html>
- [4] *Conversaciones con ChatGPT para resolver dudas específicas en la medición de tiempo, para formatear correctamente las tablas, y para corregir errores ortográficos, o de puntuación*